

WOS match upload API - integrator guide

For app developers who upload finished Warcraft III WOS matches into Punch Machine / the Discord ranking bot.

Base URL (production): <http://63.186.224.240:8787>

Protocol: HTTP JSON (HTTPS + Cloudflare planned later - see issue #163)

What this API does

1. Your app sends the raw WOS2 bot match report (same text the map/bot exports).
2. The server creates a match in status `WAITING_FOR_APPROVAL`.
3. A Discord message is posted in the league's match approval channel.
4. A match mod reviews it in Discord (can edit quitters / griefers / winner), then Approves (ratings apply) or Rejects (void).

Your app does not complete rankings. Upload only. Moderation stays in Discord.

Prerequisites (ops / staff - not the uploader app)

Someone with Discord Manage Guild must configure the target WOS league:

1. `/league_config ? set match approval channel`
2. `/league_config ? api_token ? rotate`

Copy the plaintext token once (it is not shown again). Store it as a secret in your app.

One token = one league. Do not share tokens across leagues or commit them to git.

Endpoint

`POST /v1/matches/wos-report`

Method | POST only (GET in a browser ? 404 Not Found)

Auth | Authorization: Bearer <league_api_token>

Content-Type | application/json; charset=utf-8

Max body | 1 MiB

Request body

```
{
  "reportText": "<entire raw WOS2 bot export as a single string>"
}
```

* `reportText` is required and must be a string.

* Send the full export, including the Warcraft Preload(...) wrapper if that is what the bot produces. The server extracts payload lines itself.

* Do not pre-parse into JSON players/teams. The server runs the same parser as Discord `/register_lobby` report:.

Success - 201 Created

```
{
  "matchId": "clxxxxxxxx",
  "status": "WAITING_FOR_APPROVAL",
}
```

```
"externalId": "34508754-98989487-41346570-71702689",
"suggestedWinner": 2,
"discordMessageUrl": "https://discord.com/channels/.../.../..."
}
```

Field | Type | Notes

matchId | string | Bot match id

status | string | Always WAITING_FOR_APPROVAL on success

externalId | string | Report ID\ | value=... (idempotency key)

suggestedWinner | 1 \| 2 \| null | From rounds / win flags; mod can override

discordMessageUrl | string \| null | Approval message; may be null if Discord post failed (match still saved)

Treat 201 with discordMessageUrl: null as a successful upload; staff can still approve from Discord if the message was missed.

Errors

All error bodies look like:

```
{ "error": "English message" }
```

HTTP | When | Client action

401 | Missing/invalid Authorization header or bad token | Fix token; do not retry blindly

400 | Bad JSON, missing reportText, parse/roster error, approval channel not configured, league not WOS | Fix

payload / ask ops to configure league

404 | Wrong path, or non-POST method on this path | Use exact path + POST

409 | This report externalId was already used on another non-voided match | Do not resubmit the same report; treat as already uploaded

500 | Unexpected server error | Retry with backoff; contact bot ops if persistent

Auth header

Authorization: Bearer YOUR_LEAGUE_API_TOKEN

* Scheme is Bearer (capital B), then a single space, then the token.

* No other query params or league id in the body - the token selects the league.

Example (curl)

```
BASE_URL="http://63.186.224.240:8787"
TOKEN="paste_token_from_league_config_rotate"
# report.txt = raw WOS2 export file
jq -n --rawfile t report.txt '{reportText: $t}' \
| curl -sS -X POST "$BASE_URL/v1/matches/wos-report" \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
--data-binary @-
```

Minimal shape without jq (escape carefully if the report has quotes/newlines - prefer jq or a real JSON encoder):

```
curl -sS -X POST "$BASE_URL/v1/matches/wos-report" \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"reportText": $(python3 -c 'import json,sys; print(json.dumps(sys.stdin.read()))' < report.txt)}"
```

Report format (what to put in reportText)

Expected format: WOS2_BOT_V1 export (same as in-game bot file).

Typical lines (inside Preload("...") or as plain payload):

```
ID|value=<uuid-like-id>|format=WOS2_BOT_V1|scope=MATCH
MATCH|team1_rounds=...|team2_rounds=...|players=...
PLAYER|n=...|pid=...|name=Nick#1234|team=1|win=0|left=0|...
STATS|n=...|pid=...|kills=...|deaths=...|...
ITEMS|n=...|pid=...|slot1=...|...
END|id=<same-as-ID-value>
```

Rules the server enforces:

- * format must be WOS2_BOT_V1
- * ID value and END|id= must match
- * At least one player row
- * Player names are Battle.net-style nicks; unknown nicks are created as players
- * left=1 is treated as a suggested quitter for mods

Idempotency: if you upload the same report ID twice for a live (non-cancelled) match, you get 409. Safe to treat as "already submitted."

Client implementation checklist

1. Obtain league token from staff (secret storage).
2. After a match ends, read the WOS2 bot export as UTF-8 text.
3. POST JSON { "reportText": "..." } with Bearer auth.
4. On 201, show success (optionally open/link discordMessageUrl).
5. On 409, show "already uploaded" (not a hard failure for the player).
6. On 401 / 400, surface error string to logs/UI.
7. Never use GET; never put the token in the URL or query string.
8. Timeouts: allow several seconds (Discord post happens during the request).

Out of scope for the uploader (v1)

- * Completing / voiding matches
- * Editing quitters, griefers, or winner (Discord mods only)
- * UDBR / non-WOS games
- * Multipart file upload (JSON string only)
- * OAuth / Discord user login on this endpoint